Relazione Tecnica

Blackjack Multiplayer

Architettura Client-Server in Java

e Programmazione Concorrente

Autori: Bellini Lorenzo Capasso Emanuele Croccolo Riccardo

Istituto: ITIS G. Marconi – Pontedera

Classe: 5 BI

2025/2026

Indice

1 Introduzione **3**
1.1 Contesto e Motivazioni 3
1.2 Obiettivi del Progetto 3

2 Architettura del Sistema **3**
2.1 Modello Client-Server 3
2.2 Protocollo di Comunicazione 4
2.2.1 Comandi Client → Server 4
2.2.2 Messaggi Server → Client 4
2.3 Flusso del Sistema 4

3 Analisi delle Classi **5**
3.1 ServerMain 5
3.1.1 Logica del Banco 5
3.2 AcceptThread 6
3.3 PlayerHandler 7
3.3.1 Stato del Giocatore 7
3.3.2 Ciclo di Lettura 7
3.3.3 Gestione della Mano 8
3.4 Deck e Card 8
3.4.1 Thread-Safety del Mazzo 9
3.4.2 Costruzione del Mazzo 9
3.5 Client e Reader 10
3.5.1 Separazione degli Stream I/O 10

4 Gestione della Concorrenza e Sincronizzazione **11**
4.1 CountDownLatch per la Sincronizzazione dei Turni 11
4.2 CopyOnWriteArrayList 12
4.3 AtomicInteger per l'Assegnazione dei Nomi 12
4.4 Parola Chiave "volatile" 12
4.5 Metodi "synchronized" e "notifyAll" 13

5 Logica di Gioco e Regole **13**
5.1 Calcolo del Punteggio e Gestione degli Assi 13
5.2 Natural Blackjack 14
5.3 Risoluzione del Round e Calcolo del Payout 15

6 Robustezza e Gestione delle Eccezioni **15**
6.1 Gestione delle Disconnessioni Improvvise 15

6.2 Rilevamento della Disconnessione del Client 16
6.3 Timeout sul Latch 17
6.4 Rimozione Lazy dei Giocatori Inattivi 17

7 Conclusioni 18
7.1 Valutazione del Lavoro 18

8 Collegamenti esterni 19

Sommario

Il presente documento descrive l'architettura e le scelte implementative di un'applicazione Java che simula il gioco del Blackjack in modalita' multi-giocatore, basata su un'architettura Client-Server con comunicazione via *socket* TCP. Il sistema supporta fino a quattro giocatori connessi simultaneamente, gestendo l'intera logica di gioco – dalla distribuzione delle carte alla risoluzione delle puntate – in un contesto *multithreaded*. La relazione analizza le classi principali, le primitive di sincronizzazione impiegate (`CountDownLatch`, `CopyOnWriteArrayList`, `AtomicInteger`), il protocollo testuale di comunicazione e la robustezza del sistema alla luce di disconnessioni impreviste dei client.

1. Introduzione

1.1. Contesto e Motivazioni

Il Blackjack (noto anche come *21*) e' uno dei giochi da casino' piu' diffusi al mondo. La sua struttura a turni, la necessita' di un banco centralizzato e la semplicita' delle regole lo rendono un candidato ideale per lo studio dei sistemi distribuiti. Realizzare una versione multiplayer via rete implica affrontare problemi classici dell'informatica distribuita: *sincronizzazione*, *consistenza dello stato condiviso* e *gestione dei guasti parziali*.

1.2. Obiettivi del Progetto

Gli obiettivi principali del progetto sono i seguenti:

1. Implementare un server in grado di accettare e gestire piu' client contemporaneamente tramite socket TCP.
2. Modellare la logica di dominio del gioco (mazzo, punteggi, puntate, regole del banco).
3. Garantire la sincronizzazione corretta dei turni mediante primitive di concorrenza della libreria standard Java.
4. Assicurare la robustezza del sistema in caso di disconnessione imprevista di uno o piu' client.

2. Architettura del Sistema

2.1. Modello Client-Server

Il sistema adotta un'architettura **Client-Server asimmetrica**: esiste un unico server centralizzato che detiene l'intera logica di gioco e lo stato condiviso (mazzo, mano del banco, lista dei giocatori), mentre i client hanno il ruolo di interfaccia utente, limitandosi a inviare comandi e ricevere messaggi di testo.

Questa separazione netta tra *business logic* (server) e *presentation layer* (client) e' una scelta consapevole che garantisce:

- **Integrita' del gioco**: il server e' l'arbitro unico e incontestato di tutte le decisioni;

- **Semplicita' del client**: il client e' volutamente leggero (*thin client*), non necessita di conoscere le regole del gioco;
- **Scalabilita'**: e' teoricamente possibile sostituire il client con qualsiasi strumento capace di comunicare su TCP (ad es. `telnet`).

2.2. Protocollo di Comunicazione

La comunicazione tra client e server avviene tramite un **protocollo testuale** su connessione TCP persistente. I messaggi sono sequenze di caratteri ASCII terminati da un newline (`\n`), gestite tramite `BufferedReader` e `PrintWriter`.

2.2.1. Comandi Client → Server

Comando	Descrizione
`BET <n>`	Effettua una puntata di n fiches (valido solo a inizio turno)
`HIT`	Richiede una carta aggiuntiva al mazzo
`STAY`	Termina il proprio turno mantenendo la mano corrente
`QUIT`	Disconnette il client e termina la sessione

2.2.2. Messaggi Server → Client

Il server invia messaggi in linguaggio naturale italiano, strutturati con separatori visivi ASCII (ad es. `===`) per migliorare la leggibilita' su terminale. Non e' definito un formato binario o strutturato: la scelta del protocollo testuale privilegia la *debuggability* e la semplicita' implementativa rispetto all'efficienza.

2.3. Flusso del Sistema

Di seguito e' descritto il ciclo di vita completo di una sessione di gioco:

1. **Avvio del server**: `ServerMain` apre una `ServerSocket` sulla porta 12345 e delega l'accettazione delle connessioni al thread `AcceptThread`.
2. **Connessione client**: per ogni client che si connette, `AcceptThread` instanzia un `PlayerHandler` e ne avvia l'esecuzione su un thread dedicato.
3. **Inizio del round**: `ServerMain` – nel suo ciclo principale – distribuisce due carte al banco e notifica ogni `PlayerHandler` attivo tramite `startTurn()`.

4. **Turno del giocatore**: ogni `PlayerHandler` riceve e processa i comandi del client. Il `CountDownLatch` impedisce al server di procedere finche' tutti i giocatori non hanno completato il loro turno.
5. **Turno del banco**: ultimati tutti i turni, il server applica la strategia fissa del banco (pesca fino ad un massimo di 17).
6. **Risoluzione**: per ogni giocatore viene calcolato l'esito del round e aggiornato il saldo in fiches.
7. **Nuovo round**: il ciclo ricomincia; i giocatori senza fiches o disconnessi vengono rimossi dalla lista.

3. Analisi delle Classi

3.1. ServerMain

`ServerMain` e' il punto di ingresso dell'applicazione server e detiene lo **stato globale condiviso**: la lista dei giocatori connessi (`CopyOnWriteArrayList<PlayerHandler>`), il mazzo (`Deck`) e la mano del banco (`List<Deck.Card>`).

Il metodo `main()` implementa un ciclo infinito che orchestra la sequenza dei round. La logica di filtraggio dei giocatori attivi e' esplicita:

```
List<PlayerHandler> activePlayers = new ArrayList<>();
for (PlayerHandler p : players) {
    if (p.isActive()) {
        activePlayers.add(p);
    }
}
```

Listing 1: Filtraggio dei giocatori attivi in ServerMain

3.1.1. Logica del Banco

Il banco pesca carte fintanto che il suo punteggio e' inferiore a 17, in accordo con le regole standard del Blackjack:

```
private static void dealerPlay() {
    while (handValue(dealerHand) < 17) {
        dealerHand.add(deck.draw());
    }
}
```

Listing 2: Strategia fissa del banco

Il banco non gioca affatto in caso di *natural blackjack* (Asso + carta da 10 con le prime due carte), poiche' tale condizione e' gia' vincente per definizione e viene gestita separatamente nel metodo `isNaturalBlackjack()`.

3.2. AcceptThread

`AcceptThread` estende `Thread` ed e' impostato come **thread daemon** dal `ServerMain`, il che significa che la sua sopravvivenza e' legata a quella del thread principale: quando il server termina, anche `AcceptThread` cessa automaticamente.

Il suo ciclo di lavoro e' semplice: chiama `serverSocket.accept()` in modo bloccante, verifica che il limite di giocatori non sia stato raggiunto, e – in caso positivo – crea un nuovo `PlayerHandler`:

```java
public void run() {
    while (!serverSocket.isClosed()) {
        try {
            Socket s = serverSocket.accept();
            if (players.size() >= MAX_PLAYERS) {
                PrintWriter tmpOut = new PrintWriter(s.
                    getOutputStream(), true);
                tmpOut.println("PARTITA PIENA!");
                s.close();
                continue;
            }
            String name = "Player" + playerCounter.
                getAndIncrement();
            PlayerHandler ph = new PlayerHandler(s, deck,
                name);
            players.add(ph);
            Thread playerThread = new Thread(ph);
            playerThread.start();
        } catch (Exception e) {
            if (!serverSocket.isClosed()) e.printStackTrace
                ();
        }
    }
}
```

Listing 3: Ciclo di accettazione in AcceptThread

L'uso di `playerCounter.getAndIncrement()` garantisce che l'assegnazione del nome avvenga in modo *atomico*, senza rischi di *race condition* tra connessioni quasi-simultanee.

3.3. PlayerHandler

`PlayerHandler` e' la classe piu' articolata del progetto: implementa `Runnable` e gestisce l'intera sessione di un singolo giocatore, mantenendo il suo stato interno.

3.3.1. Stato del Giocatore

Lo stato e' modellato tramite un insieme di variabili `volatile`, che garantiscono la visibilita' immediata tra thread diversi:

```
private volatile int fiches = STARTING_FICHES;  // 100 fiches iniziali
private volatile int currentBet = 0;
private volatile boolean busted = false;
private volatile boolean naturalBlackjack = false;
private volatile boolean inRound = false;
private volatile boolean active = true;
private volatile boolean myTurn = false;
```

Listing 4: Variabili di stato in PlayerHandler

La parola chiave `volatile` assicura che le scritture effettuate dal thread del giocatore siano immediatamente visibili al thread principale del server e viceversa, senza ricorrere a blocchi `synchronized` per le semplici letture di flag booleani.

3.3.2. Ciclo di Lettura

Il metodo `run()` rimane in attesa bloccante su `in.readLine()`. Ogni riga ricevuta viene instradata al metodo `handleCommand()` solo se e' il turno del giocatore; in caso contrario, il client riceve una notifica di attesa:

```
while (active) {
    String line = in.readLine();
    if (line == null) {
        disconnect();
        break;
    }
    if (!myTurn) {
        send("Non e' il tuo turno. Aspetta.");
        continue;
```

```
    }
    handleCommand(line.trim().toUpperCase());
}
```

Listing 5: Ciclo principale di lettura in PlayerHandler

Il valore `null` restituito da `readLine()` segnala la chiusura del canale di input, corrispondente alla disconnessione del client. Questo pattern e' la forma canonica di rilevamento di una connessione interrotta in Java.

3.3.3. Gestione della Mano

Il metodo `startHand()` distribuisce due carte al giocatore e gestisce immediatamente il caso del *natural blackjack*:

```
private void startHand() {
    hand.clear();
    hand.add(deck.draw());
    hand.add(deck.draw());
    int val = handValue();
    send("La tua mano: " + handToString() + "  [" + val + "]
        ");
    if (val == 21 && hand.size() == 2) {
        boolean hasAce = hand.stream().anyMatch(c -> c.
            getValue() == 11);
        boolean hasTen = hand.stream().anyMatch(c -> c.
            getValue() == 10);
        naturalBlackjack = hasAce && hasTen;
        if (naturalBlackjack) {
            send("HAI FATTO BLACKJACK!");
        }
        endTurn();
        return;
    }
    send("Digita HIT per pescare o STAY per fermarti.");
}
```

Listing 6: Distribuzione iniziale della mano in startHand()

3.4. Deck e Card

`Deck` modella un mazzo standard da 52 carte, con rigenerazione e mescolamento automatici quando le carte si esauriscono. La classe interna statica `Card` encapsula etichetta e valore numerico.

3.4.1. Thread-Safety del Mazzo

Poiche' il mazzo e' condiviso tra tutti i `PlayerHandler` (uno per thread) e dal server, tutti i metodi pubblici sono dichiarati `synchronized`, ottenendo un lock implicito sull'istanza di `Deck`:

```java
public synchronized Card draw() {
    if (cards.isEmpty()) {
        reset();  // rigenera e mescola il mazzo
    }
    return cards.remove(0);
}

public synchronized void reset() {
    cards.clear();
    // ... aggiunta delle carte ...
    Collections.shuffle(cards);
}
```

Listing 7: Thread-safety mediante synchronized in Deck

Il metodo `draw()` controlla l'esaurimento e chiama `reset()` all'interno dello stesso blocco sincronizzato, eliminando la possibilita' di una *race condition* tra il controllo (`isEmpty()`) e l'operazione (`remove(0)`).

3.4.2. Costruzione del Mazzo

Il mazzo viene costruito secondo le regole standard: le carte numeriche (2–10) hanno valore corrispondente, le figure (J, Q, K) valgono 10, e l'Asso vale 11 (con riduzione a 1 nel calcolo del punteggio, come descritto nella Sezione 5).

Carta	Valore	Copie nel mazzo
2 – 10	2 – 10	4 per valore (36 tot.)
J, Q, K	10	4 per figura (12 tot.)
A (Asso)	11*	4
Totale		52

* L'Asso e' ridotto a 1 se il totale supera 21.

3.5. Client e Reader

Il client e' volutamente minimalista, la sua architettura separa i due canali di comunicazione in due flussi di esecuzione indipendenti.

3.5.1. Separazione degli Stream I/O

```
Socket socket = new Socket(HOST, PORT);
BufferedReader in = new BufferedReader(
    new InputStreamReader(socket.getInputStream()));
BufferedReader user = new BufferedReader(
    new InputStreamReader(System.in));
PrintWriter out = new PrintWriter(socket.getOutputStream(),
    true);

Reader reader = new Reader(in);
reader.start();  // thread daemon per la lettura dal server

while ((input = user.readLine()) != null) {
    out.println(input);  // thread principale: lettura da
        tastiera
}
```

Listing 8: Avvio del thread di lettura in Client.java

Il thread `Reader` e' un thread di background che stampa a video i messaggi ricevuti dal server, mentre il thread principale del client si occupa di leggere i comandi dall'utente (`System.in`) e inoltrarli al server. Questa separazione e' fondamentale: senza di essa, la lettura bloccante da `System.in` impedirebbe la ricezione dei messaggi server (e viceversa).

4. Gestione della Concorrenza e Sincronizzazione

4.1. CountDownLatch per la Sincronizzazione dei Turni

La chaive principale per la sincronizzazione tra il thread principale del server e i thread dei giocatori e' il `CountDownLatch`. Questa classe del pacchetto `java.util.concurrent` permette a uno o piu' thread di attendere che un insieme di operazioni eseguite su altri thread si completino.

Nel contesto applicativo, il server crea un `CountDownLatch` inizializzato al numero di giocatori attivi nel round corrente:

```
CountDownLatch latch = new CountDownLatch(activePlayers.size
    ());
for (PlayerHandler p : activePlayers) {
    if (!p.isActive()) {
        latch.countDown();
        continue;
    }
    p.startTurn(latch, dealerVisible);
}

// Il server si sospende qui fino al completamento di tutti
    i turni
boolean allDone = latch.await(1, TimeUnit.MINUTES);
```

Listing 9: Uso del CountDownLatch in ServerMain

Ogni `PlayerHandler` chiama `latch.countDown()` quando il suo turno termina (per STAY, BUST, Blackjack naturale, o disconnessione):

```
public synchronized void endTurn() {
    if (!myTurn) {
        return;
    }
    myTurn = false;
    notifyAll();
    if (turnLatch != null) {
        turnLatch.countDown();
        turnLatch = null;
    }
}
```

Listing 10: Decremento del latch in PlayerHandler.endTurn()

Il parametro `TimeUnit.MINUTES` con valore 1 introduce un **timeout di guardia**: se un giocatore non risponde entro un minuto, il server forza la fine del suo turno, prevenendo un blocco indefinito.

4.2. CopyOnWriteArrayList

La lista dei giocatori e' dichiarata come `CopyOnWriteArrayList`:

```
private static final List<PlayerHandler> players =
    new CopyOnWriteArrayList<>();
```

Listing 11: Dichiarazione thread-safe della lista giocatori

`CopyOnWriteArrayList` e' una variante di `ArrayList` ottimizzata per scenari in cui le letture sono molto piu' frequenti delle scritture. Ogni operazione di modifica (`add`, `remove`) produce una **copia dell'array sottostante**, garantendo che le iterazioni in corso non vengano mai interrotte da modifiche concorrenti. In questo progetto:

- **Scritture rare**: avvengono solo alla connessione/disconnessione di un giocatore;
- **Letture frequenti**: il loop principale del server itera la lista ad ogni round.

4.3. AtomicInteger per l'Assegnazione dei Nomi

L'assegnazione progressiva dei nomi ai giocatori (`Player1`, `Player2`, ...) sfrutta `AtomicInteger`:

```
String name = "Player" + playerCounter.getAndIncrement();
```

Listing 12: Utilizzo di AtomicInteger in AcceptThread

`getAndIncrement()` e' un'operazione **atomica** a livello hardware (si basa su istruzioni Compare-And-Swap della CPU). Questo garantisce che due client connessi quasi simultaneamente ricevano sempre nomi distinti, senza ricorrere a blocchi `synchronized`.

4.4. Parola Chiave "volatile"

Il modificatore `volatile` sui campi di stato di `PlayerHandler` garantisce la **visibilita' della memoria**: ogni scrittura su un campo `volatile` e' immediatamente visibile a tutti i thread, senza ottimizzazioni del compilatore che potrebbero memorizzarla nella cache del processore. Questo e' sufficiente per i semplici flag boolea-

ni, dove non si richiede atomicita' composita ma solo propagazione immediata del valore.

4.5. Metodi "synchronized" e "notifyAll"

Il metodo `endTurn()` e' dichiarato `synchronized` per prevenire invocazioni concorrenti (ad esempio, da un timeout del server e dalla risposta del giocatore nello stesso istante). La chiamata a `notifyAll()` sveglia eventuali thread in attesa sull'oggetto, un pattern di sicurezza difensiva.

Primitiva	Scopo nel progetto
`CountDownLatch`	Sincronizzare la fine del round (attesa di tutti i turni)
`CopyOnWriteArrayList`	Iterazione concorrente sicura sulla lista giocatori
`AtomicInteger`	Contatore atomico per i nomi giocatori
`volatile`	Visibilita' immediata dei flag di stato tra thread
`synchronized`	Mutua esclusione su metodi critici di PlayerHandler

5. Logica di Gioco e Regole

5.1. Calcolo del Punteggio e Gestione degli Assi

Il calcolo del valore di una mano e' centralizzato nel metodo statico `calcHandValue()`, riutilizzabile sia per i giocatori che per il banco:

```
public static int calcHandValue(List<Deck.Card> h) {
    int total = 0;
    int aces  = 0;

    for (Deck.Card card : h) {
        total += card.getValue();     // l'Asso conta
            inizialmente 11
        if (card.getValue() == 11) {
            aces++;
        }
```

```
    }

    // Riduzione degli Assi da 11 a 1 per evitare il bust
    while (total > 21 && aces > 0) {
        total -= 10;
        aces--;
    }

    return total;
}
```

Listing 13: Algoritmo di calcolo del punteggio con gestione degli Assi

L'algoritmo e' **greedy**: assegna inizialmente il valore massimo (11) a ogni Asso, poi riduce iterativamente il valore di 10 (da 11 a 1) finche' il totale non rientra entro i 21 o gli Assi riducibili sono esauriti. Questo garantisce sempre il punteggio ottimale per il giocatore.

Esempio: una mano [A, 5, 9] viene calcolata come 11+5+9 = 25, poi ridotta a 25-10 = 15 (l'Asso vale 1). Una mano [A, A, 9] darebbe 11+11+9 = 31 → 21 (dopo due riduzioni).

5.2. Natural Blackjack

Il *natural blackjack* e' definito come la combinazione di un Asso e una carta da valore 10 (10, J, Q, K) nelle prime due carte distribuite. Sia il server (per il banco) che il `PlayerHandler` (per il giocatore) verificano questa condizione:

```
private static boolean isNaturalBlackjack(List<Deck.Card>
    hand) {
    if (hand.size() != 2) return false;
    boolean hasAce = false;
    boolean hasTen = false;
    for (Deck.Card c : hand) {
        if (c.getValue() == 11) {
            hasAce = true;
        }
        if (c.getValue() == 10) {
            hasTen = true;
        }
    }
    return hasAce && hasTen;
}
```

Listing 14: Rilevamento del natural blackjack

5.3. Risoluzione del Round e Calcolo del Payout

La risoluzione delle puntate in `resolveRound()` copre tutti i casi previsti dalle regole del Blackjack. L'ordine delle condizioni e' deliberato e rispecchia la priorita' delle regole:

1. **Bust del giocatore**: il giocatore perde indipendentemente dalla mano del banco.
2. **Blackjack del banco senza blackjack del giocatore**: il giocatore perde.
3. **Blackjack di entrambi**: pareggio, la puntata viene restituita.
4. **Blackjack del solo giocatore**: vincita con bonus (payout 2.5x).
5. **Bust del banco**: il giocatore vince.
6. **Confronto normale**: vince il punteggio piu' alto; pareggio se uguali.

Il calcolo del payout differenzia il blackjack naturale:

```java
private int payout() {
    if (naturalBlackjack) {
        return (int) Math.round(currentBet * 2.5); // 3:2 sul profitto
    }
    return currentBet * 2;  // 1:1 sul profitto
}
```

Listing 15: Calcolo del payout in PlayerHandler

Un blackjack naturale paga 3:2 (la puntata viene moltiplicata per 2.5, corrispondente a restituire la puntata + 1.5 volte come vincita), mentre una vincita normale paga 1:1 (la puntata viene raddoppiata).

6. Robustezza e Gestione delle Eccezioni

6.1. Gestione delle Disconnessioni Improvvise

La gestione della disconnessione e' centralizzata nel metodo `disconnect()` di `PlayerHandler`, che utilizza il modificatore `synchronized` e un flag booleano `active` per garantire

l'**idempotenza** dell'operazione (piu' chiamate producono lo stesso risultato senza effetti collaterali):

```java
public synchronized void disconnect() {
    if (!active) {
        return;     // guard clause: gia' disconnesso
    }
    active = false;
    myTurn = false;
    if (turnLatch != null) {
        turnLatch.countDown(); // sblocca il server se era
            in attesa
        turnLatch = null;
    }
    try {
        socket.close();
    } catch (IOException ignored) { }
}
```

Listing 16: Metodo disconnect() idempotente

Punti critici gestiti:

- La chiamata a `turnLatch.countDown()` garantisce che il server non resti bloccato in attesa di un giocatore che si e' disconnesso durante il suo turno.
- La chiusura del socket provoca la terminazione del thread di lettura di quel giocatore (la prossima chiamata a `readLine()` sollevera' un'eccezione o restituira' `null`).
- La guard clause `if (!active) return` previene la doppia decrementazione del latch, che causerebbe un contatore negativo.

6.2. Rilevamento della Disconnessione del Client

Nel ciclo di lettura di `PlayerHandler.run()`, il ritorno di `null` da `readLine()` indica la chiusura del flusso di input da parte del client (connessione terminata). Viene catturata anche `IOException` per gestire le interruzioni di rete piu' brusche:

```java
String line = in.readLine();
if (line == null) {
    disconnect();
    break;
}
// ...
```

```
} catch (IOException e) {
    disconnect();
}
```

Listing 17: Rilevamento della disconnessione in PlayerHandler.run()

6.3. Timeout sul Latch

Il server non si fida incondizionatamente dei client: il metodo `latch.await(1, TimeUnit.MINUTES)` garantisce che, anche in caso di blocco anomalo di un client, il round proceda dopo un massimo di 60 secondi. I giocatori bloccati vengono identificati tramite `isMyTurn()` e il loro turno viene forzatamente terminato:

```
boolean allDone = latch.await(1, TimeUnit.MINUTES);
if (!allDone) {
    for (PlayerHandler p : activePlayers) {
        if (p.isMyTurn()) {
            p.send("Tempo scaduto. Turno saltato
                automaticamente.");
            p.endTurn();
        }
    }
}
```

Listing 18: Gestione del timeout del round in ServerMain

6.4. Rimozione Lazy dei Giocatori Inattivi

I giocatori disconnessi o senza fiches non vengono rimossi immediatamente dalla lista, ma alla fine di ogni round tramite una fase di *cleanup*:

```
List<PlayerHandler> daRimuovere = new ArrayList<>();
for (PlayerHandler p : players) {
    if (!p.isActive()) {
        System.out.println("[SERVER] " + p.getName() + "
            rimosso.");
        daRimuovere.add(p);
    }
}
players.removeAll(daRimuovere);
```

Listing 19: Rimozione lazy dei giocatori inattivi

Questo approccio *mark-and-sweep* evita la modifica della lista durante l'iterazione (che con `CopyOnWriteArrayList` non causerebbe eccezioni ma potrebbe condurre a comportamenti incoerenti nella stessa sessione).

7. Conclusioni

7.1. Valutazione del Lavoro

Il progetto raggiunge gli obiettivi prefissati con una soluzione coerente e ben strutturata. Le principali qualita' del design sono:

- **Separazione delle responsabilita'**: ogni classe ha un ruolo preciso e delimitato (*Single Responsibility Principle*);
- **Uso corretto delle primitive di concorrenza**: la combinazione di `CountDownLatch`, `CopyOnWriteArrayList`, `AtomicInteger` e `volatile` dimostra una buona padronanza del modello di memoria Java;
- **Robustezza**: la gestione delle disconnessioni e' attenta e considera casi limite come il doppio `disconnect()`;
- **Semplicita' del client**: il thin client e' facilmente sostituibile o estendibile.

Alcune aree di miglioramento identificate:

- Il metodo `isNaturalBlackjack()` del banco controlla solo il valore 10 per la figura, ma J, Q e K hanno gia' valore 10 nel `Deck`: la logica e' corretta, ma la denominazione potrebbe causare confusione;
- La lista `dealerHand` in `ServerMain` e' condivisa senza sincronizzazione esplicita; e' sicura solo perche' il thread `AcceptThread` non la accede mai, ma questa dipendenza implicita e' fragile.

In sintesi, il progetto costituisce una solida dimostrazione delle tecniche fondamentali dei sistemi distribuiti e della programmazione concorrente in Java, con una architettura chiara e un codice ben commentato che ne facilita l'evoluzione verso scenari piu' complessi.

8. Collegamenti esterni

Il codice sorgente completo del progetto è disponibile al seguente Il progetto completo è consultabile al seguente link GitHub.